

○ QKET · 운영 문서

배포·운영 가이드

Qket 서비스의 배포·운영 기준을 한 문서에서 확인할 수 있도록 정리한 실무 가이드. 문서 목적과 사용 기술의 역할부터 GitOps 배포, 인프라 on/off, 헬스체크·시크릿·모니터링, 장애 대응 명령어, 실제 장애 사례와 롤백 절차까지 운영 중 반복적으로 필요한 기준을 담았다.

최종 갱신

2026-08-31

대상 레포 backend · frontend ·
Infra · CD

배포 방식

GitOps (ArgoCD, 수동 Sync)

RELEASE 환경	PROD 환경	배포 트리거
release → qket-release → dev.jun979.click	main → qket-prod → app.jun979.click	CI가 이미지를 만들면 ArgoCD UI에서 Sync 버튼을 눌러 실제 반영 (자동 sync는 의도적으로 꺼둠)

CONTENTS

01 문서 목적 02 사용 기술 및 용도 03 개요 04 CI/CD 파이프라인 05 인프라 구조
06 매일 운영 절차 07 헬스체크 계층 08 시크릿 관리 09 모니터링·로깅
10 알림 파이프라인 11 장애 대응 방법 12 장애 대응 사례 13 롤백 절차

01 문서 목적

purpose · scope · operating rule

이 문서는 Qket 서비스를 배포하고 운영하는 과정에서 팀원이 동일한 기준으로 작업할 수 있도록 배포 흐름, 인프라 운영 순서, 장애 확인 방법과 복구 절차를 표준화한 운영 참조 문서다. 신규 팀원이 투입되거나 장애가 발생했을 때도 개인 기억에 의존하지 않고 이 문서를 기준으로 판단하는 것을 목표로 한다.

구분	이 문서에서 제공하는 기준
배포 표준화	코드 push부터 ECR 이미지 생성, CD 레포 write-back, ArgoCD 수동 Sync까지의 책임 범위와 순서를 명확히 한다.
운영 일관성	Terraform root 적용 순서, 매일 아침·저녁 인프라 on/off, 헬스체크·시크릿·모니터링 운영 기준을 통일한다.
장애 대응	장애 발생 시 먼저 확인할 리소스, 로그·이벤트 확인 명령어, 시크릿 반영·재시작·롤백 판단 기준을 제공한다.
재발 방지	실제 발생했던 장애 사례의 증상·원인·해결을 남겨 동일 유형의 장애를 반복하지 않도록 한다.

문서 범위	대상은 backend, frontend, Infra, CD 레포와 AWS/EKS 운영 영역이다. 기능 개발 상세나 API 명세가 아니라 “배포 후 서비스를 정상 상태로 유지하고 문제를 복구하는 방법”에 초점을 둔다.
-------	--

02 사용 기술 및 용도

stack · role · ops point

Qket 운영 스택은 “애플리케이션 → CI/CD → Kubernetes/인프라 → 데이터·시크릿 → 관측·알림” 계층으로 나뉜다. 아래 표는 단순 기술 목록이 아니라 운영 시 각 기술이 맡는 역할을 기준으로 정리한 것이다.

분류	기술	주요 용도	운영 시 확인 포인트
Application	Spring Boot	backend API, Actuator 헬스체크 제공	서비스 8080 / management 8081 분리
Application	Next.js	frontend App Router, /healthz 제공	backend/DB 호출 없이 헬스체크 즉시 응답
CI	GitHub Actions	test·build, ECR push, CD 레포 write-back	GitHub App 설치 토큰으로 CD 레포 갱신
Registry	Amazon ECR	backend/frontend Docker 이미지 저장	release/main별 이미지 태그와 CD values 일치 확인
CD	ArgoCD + Helm	CD 레포 상태를 클러스터에 반영	automated sync OFF, 팀이 수동 Sync
Infra	Terraform	VPC·EKS·addon·data 리소스 IaC 관리	root 순서와 destroy 대상 명시
Kubernetes	Amazon EKS	애플리케이션·addon 실행 환경	Pod/Deployment/Service/Ingress 상태 확인
Autoscaling	Karpenter	부하에 따라 EKS 노드 생성·정리	Node/NodeClaim과 HPA 상태 함께 확인
Data	RDS / ElastiCache Redis	prod 데이터베이스와 Redis 운영	운영 on/off 루틴에서 원칙적으로 삭제 금지
Secret	Secrets Manager + ESO	비밀값 원본 보관 및 K8s Secret 동기화	Secret 변경 후 Pod 재시작까지 필요
Observability	Prometheus · Grafana · Loki · CloudWatch	지표·로그·AWS 관리형 서비스 상태 확인	namespace와 데이터소스 기준으로 원인 범위 축소
Notification	SQS · Lambda · SES	메일 이벤트 비동기 처리	SQS 메시지 → Lambda → SES 전달 흐름 확인

운영 원칙

자동화는 “반복 작업을 줄이는 범위”까지 사용하되, 실제 배포 시점과 데이터 삭제 가능성이 있는 작업은 사람이 판단한다. 특히 ArgoCD Sync와 Terraform destroy는 운영자가 상태를 확인한 뒤 실행한다.

03 개요

repo map · gitops 원칙

Qket은 다섯 개 레포로 나뉜다. 코드를 짜는 레포와 배포 상태를 나타내는 레포가 분리돼 있고, 그 사이를 ArgoCD가 감시한다. 즉, “레포에 반영된 상태 = 클러스터의 실제 상태”가 되도록 맞추는 것이 이 문서 전체를 관통하는 원칙(GitOps)이다.

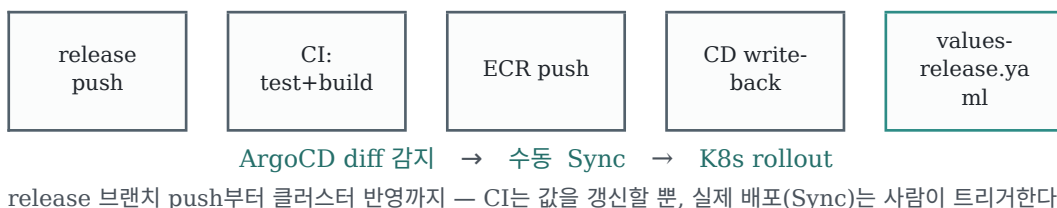
레포	역할	배포 시 하는 일
backend	Spring Boot API	release/main push → 이미지 빌드 → ECR push → CD 레포 write-back
frontend	Next.js App Router	동일 (backend와 같은 파이프라인 패턴)
Infra	Terraform (AWS 리소스 전체)	사람이 직접 terraform apply — CI로 자동화하지 않음
CD	Helm 차트 + 배포 상태 (values.yaml)	ArgoCD가 이 레포의 main을 계속 감시
CLAUDE_LLM_WIKI	팀 운영 지식 베이스	배포 대상 아님 — 결정/트러블슈팅 기록 전용

왜 코드 레포와 배포 레포를 나눴나 — backend/frontend가 직접 K8s 매니페스트를 고치게 두면, “이 이미지가 지금 뭘 배포하고 있는지”를 알기 위해 코드 레포 커밋 이력과 클러스터 상태를 동시에 봐야 한다. CD 레포 하나만 보면 release/prod 각각 어떤 이미지가 떠 있는지 바로 알 수 있도록 분리했다.

04 CI/CD 파이프라인

github actions → ecr → argocd

backend/frontend 두 레포 모두 같은 패턴을 쓴다. 브랜치가 곧 환경이고, 파이프라인 끝에서 “배포”까지 하지는 않는다. CI는 CD 레포의 상태를 갱신하는 데까지만 책임지고, 실제 클러스터 반영은 ArgoCD의 몫이다.



왜 자동 Sync를 안 켜나

ArgoCD의 automated sync(prune/selfHeal)를 켜면 CD 레포에 반영되는 즉시 클러스터가 따라간다. 현재는 일부러 꺼두고 Sync 버튼을 수동으로 누르는 방식을 쓴다. 이미지가 push된 시점과 실제로 배포하고 싶은 시점을 분리해서, 배포 타이밍을 팀이 직접 통제하기 위해서다.

write-back 인증: GitHub App

backend/frontend의 GITHUB_TOKEN은 자기 레포에만 유효해서 CD 레포를 못 건드린다. PAT 대신 qket-ci-bot GitHub App을 조직 소유로 만들어 CD 레포 하나에만 Contents: Read and write 권한을 설치했다.

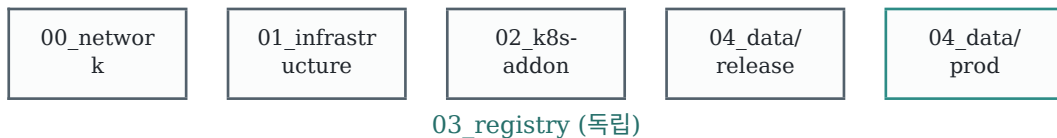
CI는 실행할 때마다 약 1시간짜리 설치 토큰을 새로 발급받아 쓰며, 저장되는 것은 앱의 private key뿐이다. 장기 자격증명을 시크릿에 두지 않는다는 원칙(AWS 인증을 OIDC로 하는 것과 같은 이유)을 따른다.

05 인프라 구조

terraform · 5-root

Infra 레포는 root 5개로 나뉘어 있고, 디렉토리 이름의 숫자 접두사가 그대로 적용(apply) 순서다. 나누는 기준은 “얼마나 자주 지웠다 켜는가”와 “release/prod가 따로 필요한가” 두 가지다.

ROOT	담는 것	성격	비용
00_network	VPC, 서브넷, 보안그룹	공유 · 단일	무료
01_infrastructu re	EKS, bastion, NAT Gateway	공유 · 단일	과금 · 매일 토글
02_k8s-addon	namespace, ArgoCD, ALB Controller, ExternalDNS, Karpenter, 모니터링 addon	공유 · 단일	과금 · 매일 토글
03_registry	ECR, GitHub Actions OIDC, ACM 인증서	공유 · env 무관	거의 무료
04_data/ release	dev-datastore(StatefulSet MySQL/Redis), S3, 앱 설정	release 전용	과금 · 절대 안 지움
04_data/prod	RDS, ElastiCache Redis, S3, 앱 설정	prod 전용	과금 · 절대 안 지움



실선은 remote_state 참조(값 의존), 점선은 독립. 최초 적용은 위→아래, destroy는 반대 순서.

최초 적용 순서 · Infra/README.md

```

# 00 → 01 → 02 → (03은 아무 때나) → 04/release → 04/prod
cd 00_network      && terraform init && terraform apply
cd 01_infrastructure && terraform init && terraform apply
cd 02_k8s-addon    && terraform init && terraform apply
cd 03_registry     && terraform init && terraform apply
cd 04_data/release  && terraform init && terraform apply
cd 04_data/prod     && terraform init && terraform apply
  
```

autoscaling · admin 접근

노드 오토스케일링은 Karpenter가 전담한다(2026-08-20, Cluster Autoscaler에서 전면 교체). Grafana·ArgoCD 콘솔과 release 도메인은 02_k8s-addon의 admin Gateway가 팀원 IP 허용목록(admin_allowed_cids)으로 접근을 제한한다. 인증 강화가 안 된 관리 콘솔을 인터넷에 그대로 노출하지 않기 위해서다.

06 매일 운영 절차

비용 절감 · on/off

01_infrastructure(EKS·bastion·NAT)와 02_k8s-addon은 비용 때문에 매일 아침 켜고 저녁에 끈다.
00_network·03_registry·04_data는 이 루틴과 무관하다. VPC/서브넷은 떠 있어도 과금되지 않고,
RDS/Redis 같은 실제 데이터는 원칙적으로 절대 지우지 않는다.

저녁 - 끄기 · 순서 중요

```
# 1. k8s-addon 먼저 (EKS Access Entry가 살아있을 때)
cd 02_k8s-addon && terraform destroy
```

```
# 2. infrastructure - 비용 나가는 리소스만 target
cd 01_infrastructure && terraform destroy \
  -target=module.eks \
  -target=module.ec2 \
  -target=aws_nat_gateway.this \
  -target=aws_eip.nat
```

아침 - 켜기 · 순서 중요

```
# 1. infrastructure 먼저 (EKS가 준비돼야 함)
cd 01_infrastructure && terraform apply
```

```
# 2. k8s-addon (namespace, ArgoCD 재생성)
cd 02_k8s-addon && terraform apply
```

```
# 3. data도 재 apply - RDS/Redis는 안 건드리고 사라진 K8s 오브젝트 3개만 다시 채움
cd 04_data/release && terraform apply
cd 04_data/prod && terraform apply
```

주의

01_infrastructure를 -target 없이 통째로 destroy하지 말 것. 로컬 .tf에 아직 반영 안 된 리소스(예: 팀원이 만든 모니터링 관련 리소스)까지 같이 지워질 수 있다. 특히 지표 이력이 쌓인 리소스가 걸리면 영구 손실. 항상 위치처럼 지울 리소스를 명시한다.

참고

02_k8s-addon이 매일 밤 destroy되면 그 안 네임스페이스가 지워지면서 04_data가 만든 K8s 오브젝트(ServiceAccount·ConfigMap)도 cascade delete된다. RDS/Redis 같은 실제 AWS 리소스는 전혀 안 건드리지만, 아침에 04_data를 다시 apply해줘야 그 오브젝트 3개가 채워진다 — 몇 초짜리 무해한 작업.

07 헬스체크 계층

alb · kubelet · actuator

같은 파드를 두 계층이 각자 다른 목적으로 체크한다. 이 둘을 같은 포트/경로로 묶어뒀던 것이 장애 사례 12-1의 원인이었다. 현재는 완전히 분리돼 있다.

계층	주체	대상	목적
ALB 헬스체크	AWS Load Balancer	Ingress healthcheck-path	이 파드에 실제 사용자 트래픽을 보내도 되는가
Liveness Probe	kubelet	:8081/actuator/health/liveness	파드가 멈췄으면 재시작
Readiness Probe	kubelet	:8081/actuator/health/readiness	준비 안 됐으면 Service 트래픽에서 제외
frontend Probe	kubelet	:3000/healthz	DB/backend 호출 없이 즉시 응답 — 일반 요청 부하와 무관

왜 actuator를 8081로 분리했나

backend는 management.server.port: 8081로 actuator를 서비스 포트(8080)와 물리적으로 분리해뒀다. 같은 포트에 있으면 실제 요청 부하가 몰릴 때 헬스체크 응답까지 같이 느려져서, 멀쩡한 파드가 “응답 없음”으로 오판돼 재시작되는 연쇄 반응이 생길 수 있다.

08 시크릿 관리

secrets manager → eso → pod

DB 비밀번호, OpenAI API 키 같은 값은 코드나 Helm values에 평문으로 두지 않는다. AWS Secrets Manager를 원본으로 두고, External Secrets Operator(ESO)가 K8s Secret으로 동기화한다.



이 구간은 자동으로 안 이어짐: Secret에 새 값이 있어도 이미 뜯 Pod는 재시작 전까지 옛 값을 계속 쓸 수 있다.

Secrets Manager 값을 바꾸는 것과 Pod에 실제로 반영되는 것은 별개 단계다. 장애 사례 12-2가 이 지점에서 발생했다.

체크리스트

Secrets Manager 값을 바꿨다면, 그것으로 끝이 아니다. ESO는 최대 1시간 뒤에야 K8s Secret을 갱신하고, 그 갱신된 Secret조차 이미 떠 있는 Pod에는 자동으로 안 들어간다(env var는 Pod 시작 시점에만 읽힘). 값을 바꾼 뒤에는 아래 명령으로 반드시 재시작까지 세트로 실행한다.

시크릿 변경 후 반영

```
kubectl rollout restart deployment/qket-backend -n qket-prod
```

09 모니터링·로깅

prometheus · grafana · loki

“지표”, “DB 상태”, “로그”를 서로 다른 도구로 나눠서 본다. 02_k8s-addon에 kube-prometheus-stack(Prometheus + Grafana + Alertmanager)과 Loki(+ Promtail)를 addon으로 올려서 클러스터 레벨 지표·로그를 수집한다.

지표	kube-prometheus-stack — 노드 CPU/메모리, 파드 상태, ALB 트래픽. 앱 커스텀 지표 (Micrometer)는 2차 범위.
로그	Loki + Promtail, 백엔드는 S3(전용 버킷). release/prod 로그는 namespace 라벨로 Grafana에서 완전히 분리 조회.
DB 상태	별도 exporter 없이 Grafana에 CloudWatch를 데이터소스로 연결 — RDS/ElastiCache 지표를 그대로 재사용.

Grafana/ArgoCD 접속은 공개 도메인으로 노출하지 않고 admin Gateway의 IP 허용목록으로만 연다(05 인프라 구조 참고).

10 알림 파이프라인

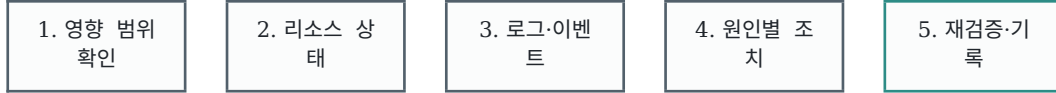
sqs → lambda → ses

회원가입 인증, 예매 확정/취소, 비밀번호 재설정, 예매 오픈 알림 — 이메일이 필요한 기능은 전부 같은 파이프라인을 공유한다. backend는 SQS에 메시지만 던지고 끝, 실제 발송은 비동기로 처리된다.

TYPE	트리거
EMAIL_VERIFICATION	회원가입 이메일 인증
RESERVATION_CONFIRMED	예매 완료
RESERVATION_CANCELLED	예매 취소
PASSWORD_RESET	비밀번호 재설정 링크

backend → SQS({"type": "...", ...}) → Lambda(notification-mailer)가 type으로 템플릿을 골라 SES로 발송. 예매 오픈 알림은 별도 Lambda(open-alert-mailer)가 담당한다.

장애가 발생하면 바로 Pod를 삭제하거나 Terraform을 다시 적용하기보다, 먼저 “어디까지 정상이고 어디서부터 실패하는지” 범위를 좁힌다. 아래 순서는 Qket에서 공통으로 사용하는 1차 트러블슈팅 절차다.



장애 대응 기본 흐름 — 증거를 먼저 확보하고, 변경은 원인이 좁혀진 뒤 최소 범위로 수행한다.

확인 단계	명령어 / 확인 위치	목적
Pod 전체 상태	<code>kubectl get pods -n qket-prod -o wide</code>	Pending / CrashLoopBackOff / Restart 증가 여부 확인
Deployment·Service	<code>kubectl get deploy,svc,ingress -n qket-prod</code>	Replica, Service, Ingress 생성 상태 확인
최근 이벤트	<code>kubectl get events -n qket-prod --sort-by=.lastTimestamp</code>	스케줄링·이미지 pull·probe 실패 등 최근 원인 확인
Pod 상세	<code>kubectl describe pod <POD_NAME> -n qket-prod</code>	Probe, Secret, Volume, Scheduling 이벤트 상세 확인
Backend 로그	<code>kubectl logs deployment/qket-backend -n qket-prod --tail=200</code>	예외·DB/Redis 연결·외부 API 오류 확인
실시간 로그	<code>kubectl logs -f deployment/qket-backend -n qket-prod</code>	재현 중 오류 흐름 실시간 추적
리소스 사용량	<code>kubectl top pods -n qket-prod / kubectl top nodes</code>	CPU·Memory 포화 여부 확인
오토스케일링	<code>kubectl get hpa -n qket-prod / kubectl get nodes</code>	Replica 증가 여부와 노드 수용 가능 여부 확인

상황별 빠른 조치

① 배포 직후 새 버전이 정상 기동하지 않을 때

```

kubectl rollout status deployment/qket-backend -n qket-prod
kubectl get pods -n qket-prod -o wide
kubectl describe pod <POD_NAME> -n qket-prod
kubectl logs deployment/qket-backend -n qket-prod --tail=200
  
```

② Secret 변경 후 OAuth·결제·DB 연결 값이 반영되지 않을 때

```

kubectl get externalsecret -n qket-prod
kubectl get secret -n qket-prod
kubectl rollout restart deployment/qket-backend -n qket-prod
kubectl rollout status deployment/qket-backend -n qket-prod
  
```


③ 트래픽 급증 / Pod 확장 지연을 확인할 때

```
kubectl get hpa -n qket-prod
kubectl top pods -n qket-prod
kubectl get nodes
kubectl get nodeclaims
```

중요

장애 원인이 확인되기 전에 무작정 terraform destroy/apply 또는 리소스 전체 삭제부터 실행하지 않는다. 로그·이벤트를 먼저 확보하고, 배포 버전 자체가 원인이라면 13 롤백 절차에 따라 ArgoCD History 또는 CD 레포 revert를 사용한다.

복구
확인

조치 후에는 Pod Running/Ready, Ingress·ALB 상태, 주요 API 응답, Grafana 지표와 오류 로그를 다시 확인한다. 장애가 종료되면 증상·원인·조치·재발 방지를 12 장애 대응 사례 형식으로 남긴다.

실제로 겪고 고친 장애 3건. 같은 실수를 반복하지 않기 위한 재발 방지 조치까지 포함한다.

ALB 헬스체크 연쇄 장애 critical

issue #28

증상	ALB가 파드를 unhealthy로 판정하면서 타깃 그룹 전체가 트래픽을 못 받는 상태로 번짐.
원인	frontend에 헬스체크 응답 엔드포인트가 없었고, backend는 actuator가 서비스 포트와 같이 물려 있어 요청 부하가 몰리면 헬스체크 응답까지 같이 느려짐 → 정상 파드가 연쇄적으로 unhealthy 판정됨.
해결	frontend에 /healthz 즉시 응답 라우트 추가, backend는 actuator를 8081로 분리하고 K8s liveness/readiness probe를 각각 연결 (07 헬스체크 계층 참고).

Google OAuth · Toss 결제 동시 장애 critical

production

증상	소셜 로그인과 결제가 같은 시점에 동시에 실패하기 시작.
원인	Secrets Manager 값은 갱신했지만, ESO의 1시간 polling 주기와 “Pod는 시작 시점에만 env를 읽는다”는 특성이 겹쳐 이미 떠 있던 Pod가 옛 시크릿 값을 계속 들고 있었음 (08 시크릿 관리 참고).
해결	kubectl rollout restart로 Pod를 재생성해 새 시크릿을 즉시 반영.

예매 오픈 시간이 “예매 전”으로 잘못 표시 warn

2026-08-21

증상	실제로는 예매가 열렸는데도 프론트엔드가 계속 “예매 전”으로 보여줌.
원인	컨테이너 기본 타임존이 UTC라, 백엔드가 Asia/Seoul 기준으로 내려준 시간 문자열을 프론트의 new Date()가 UTC로 잘못 해석.
해결	backend, frontend Deployment 둘 다 TZ=Asia/Seoul 환경변수 추가.

13 롤백 절차

argocd history · image revert

automated sync를 꺼둔 덕분에, 롤백도 “사람이 판단해서 되돌리는” 절차로 통일돼 있다. 상황에 따라 아래 두 방법 중 하나를 쓴다.

빠른 임시 롤백

ArgoCD UI → 해당 Application → History and Rollback에서 이전 리비전 선택. 클러스터만 즉시 이전 상태로 되돌아간다. CD 레포 상태는 그대로이므로, 다음에 아무 커밋이든 Sync를 누르면 다시 문제 버전으로 돌아갈 수 있다 — 원인 수정 전까지의 임시 조치로만 쓴다.

근본 롤백

CD 레포의 write-back 커밋을 git revert(또는 helm/values-*.yaml의 image 태그를 이전 값으로 직접 수정)한 뒤 push → ArgoCD가 diff를 감지 → Sync 버튼으로 반영. 레포 상태 자체가 되돌아가므로 이후 Sync에도 안전하다.

Qket 운영 원칙 · 코드/배포 상태 분리 · 수동 Sync · 데이터 보존 · 헬스체크 분리 · 시크릿 변경 후 Pod 재시작